Lesson 11 Teacher's Key: Wave After Wave

GameMaker - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

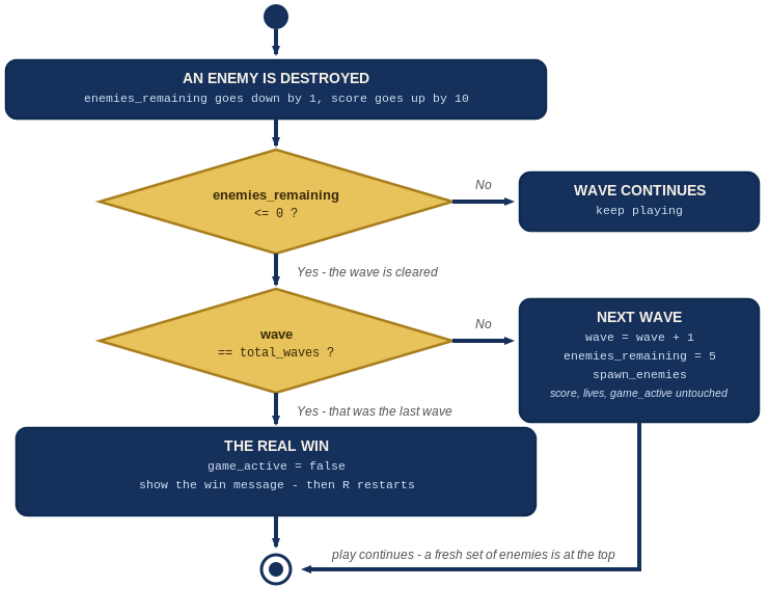
Giving the spawn loop its own named function (spawn_enemies) so start_game() and the win check can both call it - GameMaker's version of 'write it once, call it from more than one place'.

Level 1 - Full Worked Solution

- In Lesson 10 the spawn loop lived inside start_game(). This week it moves into spawn_enemies() in its own Script resource, and start_game() calls it. The enemy-creating code still exists once.
- wave and total_waves live on obj_gamemanager (Rule 13), next to enemies_remaining. The bullet reaches them with the obj_gamemanager. prefix, exactly as it does for score.
- spawn_enemies() is a global function, so obj_bullet can call it with no prefix: the same call as start_game() makes.
- GML Visual path: the same three Function Call blocks and one Declare a New Function, with every Set Variable block's Target on obj_gamemanager. The wave check is a nested If Variable plus an Else (Rule 7 detail in the Handout and the diagram above).

One Enemy Down - Is the Wave Over? Is the Game?

The same shape in every engine, whatever the exact code or blocks look like



spawn_enemies is the ONE routine that creates a wave: launch, restart and every new wave all call it. The enemy-creating code still exists only once.

VIDEO GAME CREATION

Clearing a wave asks a second question. If it was not the last wave, the game spawns a fresh set of enemies and carries on. The same shape in every engine.



scr_spawn_enemies, scr_start_game, obj_gamemanager and obj_bullet - the block-for-block equivalent of the Level 1 code. (the Visual path)

Why This Structure

- Introducing a second Script resource is deliberate: it reinforces last week's idea that scripts hold reusable functions, and keeps start_game() short enough to read at a glance.
- Using the same names in Code and Visual (spawn_enemies, enemy_count, wave, total_waves) lets students compare the two paths side by side (Rules 11 and 12).

Level 2 / Level 3 Talking Points

- Ask students to name every caller of spawn_enemies() (start_game() and the else branch of the bullet's win check) and what triggers each.

- Let students predict: what happens to wave if the bullet's else branch forgets obj_gamemanager.? (A variable-not-set error on the bullet, since wave lives on the manager.)
- The Stalemate (Level 3): do NOT hand over the fix. It was Lesson 10's Level 3 and is deliberately left in so it becomes irritating enough that students go back and fix it. Waves make it worse: after any escape, enemies_remaining can never reach 0, so that wave never ends - at any of the three waves. Confirm it is real if a student reports it, name it (the Stalemate), and nudge: 'what has to be true for a wave to be over?' The worked fix below makes an escape also resolve an enemy, and routes both the kill path and the escape path through ONE wave-cleared decision. Accept any design that guarantees every wave ends in exactly one outcome. Test: 3 kills and 2 escapes, 4 and 1, 0 and 5 (the last should be a loss only if lives run out), in each of the three waves.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: (1) clear all five three times: waves 1 and 2 continue, wave 3
shows "You Win!";
// (2) lose the last life in wave 2: "Game Over", no new wave;
// (3) R after a win and after a loss: wave 1, score 0, lives 3, exactly
five enemies.
```

Level 3 - complete code (additions on top of Level 1 highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code. Methods shown are the final versions of everything Level 3 touches.

```
// scr_start_game - Level 3 (final versions of everything Level 3
touches):
function start_game() {
    with (obj_enemy) {
        instance_destroy();
    }
    obj_gamemanager.__dnd_score = 0;
    obj_gamemanager.__dnd_lives = 3;
    obj_gamemanager.wave = 1;
    obj_gamemanager.enemies_remaining = 5;
    obj_gamemanager.has_won = false;
    obj_gamemanager.has_lost = false;
    obj_gamemanager.wave_message_timer = 0;
    obj_gamemanager.game_active = true;
    spawn_enemies();
}

function wave_cleared() {
    if (obj_gamemanager.wave >= obj_gamemanager.total_waves) {
        obj_gamemanager.has_won = true;
        obj_gamemanager.game_active = false;
        audio_play_sound(sfx_win, 1, false);  // your Lesson 6 Level 3
sound
    } else {
        obj_gamemanager.wave += 1;
        obj_gamemanager.enemies_remaining = 5;
        obj_gamemanager.wave_message_timer =
game_get_speed(gamespeed_fps);
        spawn_enemies();
```

```
    }
}

// obj_gamemanager's Create event (add):
wave_message_timer = 0;

// obj_gamemanager's Step event:
if (!game_active && keyboard_check_pressed(ord("R"))) {
    start_game();
}
if (wave_message_timer > 0) {
    wave_message_timer -= 1;
}

// obj_gamemanager's Draw GUI event (add, after the existing lines):
if (wave_message_timer > 0) {
    draw_text(400, 200, "Wave " + string(wave) + "!");
}

// obj_bullet's Collision event with obj_enemy:
audio_play_sound(sfx_hit, 1, false);
obj_gamemanager.enemies_remaining -= 1;
obj_gamemanager.__dnd_score += 10;
if (obj_gamemanager.enemies_remaining <= 0) {
    wave_cleared();
}
instance_destroy(other);
instance_destroy(self);

// obj_enemy's Step event:
if (!obj_gamemanager.game_active) exit;
y += enemy_speed;
if (y > room_height) {
    obj_gamemanager.__dnd_lives -= 1;
    obj_gamemanager.enemies_remaining -= 1;
    if (obj_gamemanager.__dnd_lives <= 0) {
        obj_gamemanager.has_lost = true;
        obj_gamemanager.game_active = false;
        audio_play_sound(sfx_gameover, 1, false);  // your Lesson 8 Level
3 sound
    } else if (obj_gamemanager.enemies_remaining <= 0) {
        wave_cleared();
    } else {
        audio_play_sound(sfx_lifelost, 1, false);  // your Lesson 8 Level
3 sound
    }
    instance_destroy();
}

// GML Visual: put wave_cleared in a "Declare a New Function" block in
scr_start_game holding the nested If Variable / Else stack from the
bullet's Level 1 blocks; replace those blocks in obj_bullet's Collision
event with a Function Call (Function = wave_cleared); in obj_enemy's
escape branch add a Set Variable (Target obj_gamemanager,
enemies_remaining, Value -1, Relative checked) after the Set Lives block,
then an Else after the lives If with an If Variable (enemies_remaining <=
```

```
0) holding a Function Call (Function = wave_cleared). The announcement is
a wave_message_timer Set Variable, a Step If Variable counting down, and
an If Variable plus Draw Text in Draw GUI.
```

Common Student Mistakes

- Game-state drift: wave not reset to 1 in start_game(), so a restart after wave 3 begins at wave 3.
- Forgetting the obj_gamemanager. prefix on wave, total_waves and enemies_remaining inside obj_bullet.
- Visual: using Execute Script instead of Function Call for spawn_enemies, or leaving the spawn-loop blocks outside the Declare a New Function block (layer-does-not-exist error at game start).
- Checking only enemies_remaining <= 0 without the wave comparison, so clearing wave 1 immediately wins.
- A student reports a wave that never ends: that is the Stalemate. Do not hand over the fix.